Sign in

pytorch / pytorch Public

Notifications Fork 23.6k Star 87.8k

Code Issues 5k+ Pull requests 1.1k Actions Projects 12

v2.6.0

pytorch / torch / nn / modules

Go to file

/ sparse.py

VladimirFokow and pytorchmergebot doc - fix the max_norm value in a note (#129687) 6adc725 · 8 months ago

546 lines (485 loc) · 23.5 KB .

Code Blame

Raw

```
2   from typing import Optional
3
4   import torch
5   from torch import Tensor
6   from torch.nn import functional as F, init
7   from torch.nn.parameter import Parameter
8
9   from .module import Module
10
11
12  __all__ = ["Embedding", "EmbeddingBag"]
13
```

```
class Embedding(Module):
    r"""A simple lookup table that stores embeddings of a fixed dictionary and size.

    This module is often used to store word embeddings and retrieve them using indices.
    The input to the module is a list of indices, and the output is the corresponding
    word embeddings.

    Args:
        num_embeddings (int): size of the dictionary of embeddings
        embedding_dim (int): the size of each embedding vector
        padding_idx (int, optional): If specified, the entries at :attr:`padding_idx` do not contribute to the gradient;
                                     therefore, the embedding vector at :attr:`padding_idx` is not updated during training,
                                     i.e. it remains as a fixed "pad". For a newly constructed Embedding,
                                     the embedding vector at :attr:`padding_idx` will default to all zeros,
                                     but can be updated to another value to be used as the padding vector.
        max_norm (float, optional): If given, each embedding vector with norm larger than :attr:`max_norm`
                                    is renormalized to have norm :attr:`max_norm`.
        norm_type (float, optional): The p of the p-norm to compute for the :attr:`max_norm` option. Default ``2``.
        scale_grad_by_freq (bool, optional): If given, this will scale gradients by the inverse of frequency of
                                             the words in the mini-batch. Default ``False``.
        sparse (bool, optional): If ``True``, gradient w.r.t. :attr:`weight` matrix will be a sparse tensor.
                                 See Notes for more details regarding sparse gradients.

    Attributes:
        weight (Tensor): the learnable weights of the module of shape (num_embeddings, embedding_dim)
                         initialized from :math:`\mathcal{N}(0, 1)`

    Shape:
        - Input: :math:`(*)`, IntTensor or LongTensor of arbitrary shape containing the indices to extract
        - Output: :math:`(*, H)`, where `*` is the input shape and :math:`H=\text{embedding\_dim}`

    .. note::
        Keep in mind that only a limited number of optimizers support
        sparse gradients: currently it's :class:`optim.SGD` (`CUDA` and `CPU`),
        :class:`optim.SparseAdam` (`CUDA` and `CPU`) and :class:`optim.Adagrad` (`CPU`)

    .. note::
```

```
        When :attr:`max_norm` is not ``None``, :class:`Embedding`'s forward method will modify the
        :attr:`weight` tensor in-place. Since tensors needed for gradient computations cannot be
        modified in-place, performing a differentiable operation on ``Embedding.weight`` before
        calling :class:`Embedding`'s forward method requires cloning ``Embedding.weight`` when
        :attr:`max_norm` is not ``None``. For example::

            n, d, m = 3, 5, 7
            embedding = nn.Embedding(n, d, max_norm=1.0)
            W = torch.randn((m, d), requires_grad=True)
            idx = torch.tensor([1, 2])
            a = embedding.weight.clone() @ W.t()  # weight must be cloned for this to be differentiable
            b = embedding(idx) @ W.t()  # modifies weight in-place
            out = (a.unsqueeze(0) + b.unsqueeze(1))
            loss = out.sigmoid().prod()
            loss.backward()

    Examples::

        >>> # an Embedding module containing 10 tensors of size 3
        >>> embedding = nn.Embedding(10, 3)
        >>> # a batch of 2 samples of 4 indices each
        >>> input = torch.LongTensor([[1, 2, 4, 5], [4, 3, 2, 9]])
        >>> # xdoctest: +IGNORE_WANT("non-deterministic")
        >>> embedding(input)
        tensor([[[-0.0251, -1.6902,  0.7172],
                 [-0.6431,  0.0748,  0.6969],
                 [ 1.4970,  1.3448, -0.9685],
                 [-0.3677, -2.7265, -0.1685]],

                [[ 1.4970,  1.3448, -0.9685],
                 [ 0.4362, -0.4004,  0.9400],
                 [-0.6431,  0.0748,  0.6969],
                 [ 0.9124, -2.3616,  1.1151]]])


        >>> # example with padding_idx
        >>> embedding = nn.Embedding(10, 3, padding_idx=0)
        >>> input = torch.LongTensor([[0, 2, 0, 5]])
        >>> embedding(input)
```

```
            tensor([[[ 0.0000,  0.0000,  0.0000],
                     [ 0.1535, -2.0309,  0.9315],
                     [ 0.0000,  0.0000,  0.0000],
                     [-0.1655,  0.9897,  0.0635]]])

            >>> # example of changing `pad` vector
            >>> padding_idx = 0
            >>> embedding = nn.Embedding(3, 3, padding_idx=padding_idx)
            >>> embedding.weight
            Parameter containing:
            tensor([[ 0.0000,  0.0000,  0.0000],
                    [-0.7895, -0.7089, -0.0364],
                    [ 0.6778,  0.5803,  0.2678]], requires_grad=True)
            >>> with torch.no_grad():
            ...     embedding.weight[padding_idx] = torch.ones(3)
            >>> embedding.weight
            Parameter containing:
            tensor([[ 1.0000,  1.0000,  1.0000],
                    [-0.7895, -0.7089, -0.0364],
                    [ 0.6778,  0.5803,  0.2678]], requires_grad=True)
        """

        __constants__ = [
            "num_embeddings",
            "embedding_dim",
            "padding_idx",
            "max_norm",
            "norm_type",
            "scale_grad_by_freq",
            "sparse",
        ]

        num_embeddings: int
        embedding_dim: int
        padding_idx: Optional[int]
        max_norm: Optional[float]
        norm_type: float
        scale_grad_by_freq: bool
```

```
129         weight: Tensor
130         freeze: bool
131         sparse: bool
132
133         def __init__(
134             self,
135             num_embeddings: int,
136             embedding_dim: int,
137             padding_idx: Optional[int] = None,
138             max_norm: Optional[float] = None,
139             norm_type: float = 2.0,
140             scale_grad_by_freq: bool = False,
141             sparse: bool = False,
142             _weight: Optional[Tensor] = None,
143             _freeze: bool = False,
144             device=None,
145             dtype=None,
146         ) -> None:
147             factory_kwargs = {"device": device, "dtype": dtype}
148             super().__init__()
149             self.num_embeddings = num_embeddings
150             self.embedding_dim = embedding_dim
151             if padding_idx is not None:
152                 if padding_idx > 0:
153                     assert (
154                         padding_idx < self.num_embeddings
155                     ), "Padding_idx must be within num_embeddings"
156                 elif padding_idx < 0:
157                     assert (
158                         padding_idx >= -self.num_embeddings
159                     ), "Padding_idx must be within num_embeddings"
160                     padding_idx = self.num_embeddings + padding_idx
161             self.padding_idx = padding_idx
162             self.max_norm = max_norm
163             self.norm_type = norm_type
164             self.scale_grad_by_freq = scale_grad_by_freq
165             if _weight is None:
166                 self.weight = Parameter(
```

```
167                     torch.empty((num_embeddings, embedding_dim), **factory_kwargs),
168                     requires_grad=not _freeze,
169                 )
170                 self.reset_parameters()
171             else:
172                 assert list(_weight.shape) == [
173                     num_embeddings,
174                     embedding_dim,
175                 ], "Shape of weight does not match num_embeddings and embedding_dim"
176                 self.weight = Parameter(_weight, requires_grad=not _freeze)
177 
178             self.sparse = sparse
179 
180         def reset_parameters(self) -> None:
181             init.normal_(self.weight)
182             self._fill_padding_idx_with_zero()
183 
184         def _fill_padding_idx_with_zero(self) -> None:
185             if self.padding_idx is not None:
186                 with torch.no_grad():
187                     self.weight[self.padding_idx].fill_(0)
188 
189         def forward(self, input: Tensor) -> Tensor:
190             return F.embedding(
191                 input,
192                 self.weight,
193                 self.padding_idx,
194                 self.max_norm,
195                 self.norm_type,
196                 self.scale_grad_by_freq,
197                 self.sparse,
198             )
199 
200         def extra_repr(self) -> str:
201             s = "{num_embeddings}, {embedding_dim}"
202             if self.padding_idx is not None:
203                 s += ", padding_idx={padding_idx}"
204             if self.max_norm is not None:
205                 s += ", max_norm={max_norm}"
```

```
206             if self.norm_type != 2:
207                 s += ", norm_type={norm_type}"
208             if self.scale_grad_by_freq is not False:
209                 s += ", scale_grad_by_freq={scale_grad_by_freq}"
210             if self.sparse is not False:
211                 s += ", sparse=True"
212             return s.format(**self.__dict__)
213
214         @classmethod
215         def from_pretrained(
216             cls,
217             embeddings,
218             freeze=True,
219             padding_idx=None,
220             max_norm=None,
221             norm_type=2.0,
222             scale_grad_by_freq=False,
223             sparse=False,
224         ):
225             r"""Create Embedding instance from given 2-dimensional FloatTensor.
226
227             Args:
228                 embeddings (Tensor): FloatTensor containing weights for the Embedding.
229                     First dimension is being passed to Embedding as ``num_embeddings``, second as ``embedding_dim``.
230                 freeze (bool, optional): If ``True``, the tensor does not get updated in the learning process.
231                     Equivalent to ``embedding.weight.requires_grad = False``. Default: ``True``
232                 padding_idx (int, optional): If specified, the entries at :attr:`padding_idx` do not contribute to the gradient;
233                                              therefore, the embedding vector at :attr:`padding_idx` is not updated during training,
234                                              i.e. it remains as a fixed "pad".
235                 max_norm (float, optional): See module initialization documentation.
236                 norm_type (float, optional): See module initialization documentation. Default ``2``.
237                 scale_grad_by_freq (bool, optional): See module initialization documentation. Default ``False``.
238                 sparse (bool, optional): See module initialization documentation.
239
240             Examples::
241
242                 >>> # FloatTensor containing pretrained weights
243                 >>> weight = torch.FloatTensor([[1, 2.3, 3], [4, 5.1, 6.3]])
```

```
            >>> embedding = nn.Embedding.from_pretrained(weight)
            >>> # Get embeddings for index 1
            >>> input = torch.LongTensor([1])
            >>> # xdoctest: +IGNORE_WANT("non-deterministic")
            >>> embedding(input)
            tensor([[ 4.0000,  5.1000,  6.3000]])
        """
        assert (
            embeddings.dim() == 2
        ), "Embeddings parameter is expected to be 2-dimensional"
        rows, cols = embeddings.shape
        embedding = cls(
            num_embeddings=rows,
            embedding_dim=cols,
            _weight=embeddings,
            _freeze=freeze,
            padding_idx=padding_idx,
            max_norm=max_norm,
            norm_type=norm_type,
            scale_grad_by_freq=scale_grad_by_freq,
            sparse=sparse,
        )
        return embedding


class EmbeddingBag(Module):
    r"""Compute sums or means of 'bags' of embeddings, without instantiating the intermediate embeddings.

    For bags of constant length, no :attr:`per_sample_weights`, no indices equal to :attr:`padding_idx`,
    and with 2D inputs, this class

        * with ``mode="sum"`` is equivalent to :class:`~torch.nn.Embedding` followed by ``torch.sum(dim=1)``,
        * with ``mode="mean"`` is equivalent to :class:`~torch.nn.Embedding` followed by ``torch.mean(dim=1)``,
        * with ``mode="max"`` is equivalent to :class:`~torch.nn.Embedding` followed by ``torch.max(dim=1)``.

    However, :class:`~torch.nn.EmbeddingBag` is much more time and memory efficient than using a chain of these
    operations.

    EmbeddingBag also supports per_sample_weights as an argument to the forward
```

```
        EmbeddingBag also supports per-sample weights as an argument to the forward
        pass. This scales the output of the Embedding before performing a weighted
        reduction as specified by ``mode``. If :attr:`per_sample_weights` is passed, the
        only supported ``mode`` is ``"sum"``, which computes a weighted sum according to
        :attr:`per_sample_weights`.

        Args:
            num_embeddings (int): size of the dictionary of embeddings
            embedding_dim (int): the size of each embedding vector
            max_norm (float, optional): If given, each embedding vector with norm larger than :attr:`max_norm`
                                        is renormalized to have norm :attr:`max_norm`.
            norm_type (float, optional): The p of the p-norm to compute for the :attr:`max_norm` option. Default ``2``.
            scale_grad_by_freq (bool, optional): if given, this will scale gradients by the inverse of frequency of
                                                 the words in the mini-batch. Default ``False``.
                                                 Note: this option is not supported when ``mode="max"``.
            mode (str, optional): ``"sum"``, ``"mean"`` or ``"max"``. Specifies the way to reduce the bag.
                                  ``"sum"`` computes the weighted sum, taking :attr:`per_sample_weights`
                                  into consideration. ``"mean"`` computes the average of the values
                                  in the bag, ``"max"`` computes the max value over each bag.
                                  Default: ``"mean"``
            sparse (bool, optional): if ``True``, gradient w.r.t. :attr:`weight` matrix will be a sparse tensor. See
                                     Notes for more details regarding sparse gradients. Note: this option is not
                                     supported when ``mode="max"``.
            include_last_offset (bool, optional): if ``True``, :attr:`offsets` has one additional element, where the last element
                                                  is equivalent to the size of `indices`. This matches the CSR format.
            padding_idx (int, optional): If specified, the entries at :attr:`padding_idx` do not contribute to the
                                         gradient; therefore, the embedding vector at :attr:`padding_idx` is not updated
                                         during training, i.e. it remains as a fixed "pad". For a newly constructed
                                         EmbeddingBag, the embedding vector at :attr:`padding_idx` will default to all
                                         zeros, but can be updated to another value to be used as the padding vector.
                                         Note that the embedding vector at :attr:`padding_idx` is excluded from the
                                         reduction.

        Attributes:
            weight (Tensor): the learnable weights of the module of shape `(num_embeddings, embedding_dim)`
                             initialized from :math:`\mathcal{N}(0, 1)`.

        Examples::
```

```
321             >>> # an EmbeddingBag module containing 10 tensors of size 3
322             >>> embedding_sum = nn.EmbeddingBag(10, 3, mode='sum')
323             >>> # a batch of 2 samples of 4 indices each
324             >>> input = torch.tensor([1, 2, 4, 5, 4, 3, 2, 9], dtype=torch.long)
325             >>> offsets = torch.tensor([0, 4], dtype=torch.long)
326             >>> # xdoctest: +IGNORE_WANT("non-deterministic")
327             >>> embedding_sum(input, offsets)
328             tensor([[-0.8861, -5.4350, -0.0523],
329                     [ 1.1306, -2.5798, -1.0044]])
330
331             >>> # Example with padding_idx
332             >>> embedding_sum = nn.EmbeddingBag(10, 3, mode='sum', padding_idx=2)
333             >>> input = torch.tensor([2, 2, 2, 2, 4, 3, 2, 9], dtype=torch.long)
334             >>> offsets = torch.tensor([0, 4], dtype=torch.long)
335             >>> embedding_sum(input, offsets)
336             tensor([[ 0.0000,  0.0000,  0.0000],
337                     [-0.7082,  3.2145, -2.6251]])
338
339             >>> # An EmbeddingBag can be loaded from an Embedding like so
340             >>> embedding = nn.Embedding(10, 3, padding_idx=2)
341             >>> embedding_sum = nn.EmbeddingBag.from_pretrained(
342                     embedding.weight,
343                     padding_idx=embedding.padding_idx,
344                     mode='sum')
345         """
346
347         __constants__ = [
348             "num_embeddings",
349             "embedding_dim",
350             "max_norm",
351             "norm_type",
352             "scale_grad_by_freq",
353             "mode",
354             "sparse",
355             "include_last_offset",
356             "padding_idx",
357         ]
358
```

```
        num_embeddings: int
        embedding_dim: int
        max_norm: Optional[float]
        norm_type: float
        scale_grad_by_freq: bool
        weight: Tensor
        mode: str
        sparse: bool
        include_last_offset: bool
        padding_idx: Optional[int]

        def __init__(
            self,
            num_embeddings: int,
            embedding_dim: int,
            max_norm: Optional[float] = None,
            norm_type: float = 2.0,
            scale_grad_by_freq: bool = False,
            mode: str = "mean",
            sparse: bool = False,
            _weight: Optional[Tensor] = None,
            include_last_offset: bool = False,
            padding_idx: Optional[int] = None,
            device=None,
            dtype=None,
        ) -> None:
            factory_kwargs = {"device": device, "dtype": dtype}
            super().__init__()
            self.num_embeddings = num_embeddings
            self.embedding_dim = embedding_dim
            self.max_norm = max_norm
            self.norm_type = norm_type
            self.scale_grad_by_freq = scale_grad_by_freq
            if padding_idx is not None:
                if padding_idx > 0:
                    assert (
                        padding_idx < self.num_embeddings
                    ), "padding_idx must be within num_embeddings"
```

```
                elif padding_idx < 0:
                    assert (
                        padding_idx >= -self.num_embeddings
                    ), "padding_idx must be within num_embeddings"
                    padding_idx = self.num_embeddings + padding_idx
            self.padding_idx = padding_idx
            if _weight is None:
                self.weight = Parameter(
                    torch.empty((num_embeddings, embedding_dim), **factory_kwargs)
                )
                self.reset_parameters()
            else:
                assert list(_weight.shape) == [
                    num_embeddings,
                    embedding_dim,
                ], "Shape of weight does not match num_embeddings and embedding_dim"
                self.weight = Parameter(_weight)
            self.mode = mode
            self.sparse = sparse
            self.include_last_offset = include_last_offset

        def reset_parameters(self) -> None:
            init.normal_(self.weight)
            self._fill_padding_idx_with_zero()

        def _fill_padding_idx_with_zero(self) -> None:
            if self.padding_idx is not None:
                with torch.no_grad():
                    self.weight[self.padding_idx].fill_(0)

        def forward(
            self,
            input: Tensor,
            offsets: Optional[Tensor] = None,
            per_sample_weights: Optional[Tensor] = None,
        ) -> Tensor:
            """Forward pass of EmbeddingBag.

            Args:
```

```
436             input (Tensor): Tensor containing bags of indices into the embedding matrix.
437             offsets (Tensor, optional): Only used when :attr:`input` is 1D. :attr:`offsets` determines
438                 the starting index position of each bag (sequence) in :attr:`input`.
439             per_sample_weights (Tensor, optional): a tensor of float / double weights, or None
440                 to indicate all weights should be taken to be ``1``. If specified, :attr:`per_sample_weights`
441                 must have exactly the same shape as input and is treated as having the same
442                 :attr:`offsets`, if those are not ``None``. Only supported for ``mode='sum'``.
443
444         Returns:
445             Tensor output shape of `(B, embedding_dim)`.
446
447         .. note::
448
449             A few notes about ``input`` and ``offsets``:
450
451             - :attr:`input` and :attr:`offsets` have to be of the same type, either int or long
452
453             - If :attr:`input` is 2D of shape `(B, N)`, it will be treated as ``B`` bags (sequences)
454               each of fixed length ``N``, and this will return ``B`` values aggregated in a way
455               depending on the :attr:`mode`. :attr:`offsets` is ignored and required to be ``None`` in this case.
456
457             - If :attr:`input` is 1D of shape `(N)`, it will be treated as a concatenation of
458               multiple bags (sequences).  :attr:`offsets` is required to be a 1D tensor containing the
459               starting index positions of each bag in :attr:`input`. Therefore, for :attr:`offsets` of shape `(B)`,
460               :attr:`input` will be viewed as having ``B`` bags. Empty bags (i.e., having 0-length) will have
461               returned vectors filled by zeros.
462         """
463         return F.embedding_bag(
464             input,
465             self.weight,
466             offsets,
467             self.max_norm,
468             self.norm_type,
469             self.scale_grad_by_freq,
470             self.mode,
471             self.sparse,
472             per_sample_weights,
473             self.include_last_offset,
```

```
            self.padding_idx,
        )

    def extra_repr(self) -> str:
        s = "{num_embeddings}, {embedding_dim}"
        if self.max_norm is not None:
            s += ", max_norm={max_norm}"
        if self.norm_type != 2:
            s += ", norm_type={norm_type}"
        if self.scale_grad_by_freq is not False:
            s += ", scale_grad_by_freq={scale_grad_by_freq}"
        s += ", mode={mode}"
        if self.padding_idx is not None:
            s += ", padding_idx={padding_idx}"
        return s.format(**{k: repr(v) for k, v in self.__dict__.items()})

    @classmethod
    def from_pretrained(
        cls,
        embeddings: Tensor,
        freeze: bool = True,
        max_norm: Optional[float] = None,
        norm_type: float = 2.0,
        scale_grad_by_freq: bool = False,
        mode: str = "mean",
        sparse: bool = False,
        include_last_offset: bool = False,
        padding_idx: Optional[int] = None,
    ) -> "EmbeddingBag":
        r"""Create EmbeddingBag instance from given 2-dimensional FloatTensor.

        Args:
            embeddings (Tensor): FloatTensor containing weights for the EmbeddingBag.
                First dimension is being passed to EmbeddingBag as 'num_embeddings', second as 'embedding_dim'.
            freeze (bool, optional): If ``True``, the tensor does not get updated in the learning process.
                Equivalent to ``embeddingbag.weight.requires_grad = False``. Default: ``True``
            max_norm (float, optional): See module initialization documentation. Default: ``None``
            norm_type (float, optional): See module initialization documentation. Default ``2``.
            scale_grad_by_freq (bool, optional): See module initialization documentation. Default ``False``.
```

```
            scale_grad_by_freq (bool, optional): See module initialization documentation. Default: ``False``.
            mode (str, optional): See module initialization documentation. Default: ``"mean"``
            sparse (bool, optional): See module initialization documentation. Default: ``False``.
            include_last_offset (bool, optional): See module initialization documentation. Default: ``False``.
            padding_idx (int, optional): See module initialization documentation. Default: ``None``.

        Examples::

            >>> # FloatTensor containing pretrained weights
            >>> weight = torch.FloatTensor([[1, 2.3, 3], [4, 5.1, 6.3]])
            >>> embeddingbag = nn.EmbeddingBag.from_pretrained(weight)
            >>> # Get embeddings for index 1
            >>> input = torch.LongTensor([[1, 0]])
            >>> # xdoctest: +IGNORE_WANT("non-deterministic")
            >>> embeddingbag(input)
            tensor([[ 2.5000,  3.7000,  4.6500]])
        """
        assert (
            embeddings.dim() == 2
        ), "Embeddings parameter is expected to be 2-dimensional"
        rows, cols = embeddings.shape
        embeddingbag = cls(
            num_embeddings=rows,
            embedding_dim=cols,
            _weight=embeddings,
            max_norm=max_norm,
            norm_type=norm_type,
            scale_grad_by_freq=scale_grad_by_freq,
            mode=mode,
            sparse=sparse,
            include_last_offset=include_last_offset,
            padding_idx=padding_idx,
        )
        embeddingbag.weight.requires_grad = not freeze
        return embeddingbag
```